PROGRAMACIÓN AVANZADA · UTU · BACHILLERATO TECNOLÓGICO

Actividades de práctica — Herencia en la Base de Datos

Tres actividades para llevar la herencia de Java a MySQL: una tabla hija con clave foránea, consultas con **INNER JOIN** y transacciones para escribir en dos tablas a la vez. Primero con la clase Profesor y después con una subclase de la entidad propia.

TEMA 4 — HERENCIA + JDBC | INNER JOIN · TRANSACCIONES | ACT. 03 — ENTIDAD PROPIA | DIFICULTAD CRECIENTE

CONTEXTO

El instituto necesita registrar a sus **profesores**. Un profesor **es una persona**: tiene código, nombre y sueldo, y además tiene una **especialidad** (la materia que dicta) y una cantidad de **horas semanales**. En lugar de repetir los datos de la persona, la clase `ProfesorVO` hereda de `PersonaVO`, y en MySQL la tabla `profesores` guarda solo los datos propios, unida a `personas` por el código.

Se trabaja sobre el proyecto terminado de la **Práctica 4** (Actividades 01 y 02). `PersonaVO`, `PersonaDAO` y `PersonaControladorServlet` no se modifican: el profesor se agrega con clases nuevas.

El modelo: herencia en Java y en MySQL

```
JAVA                                    MYSQL

PersonaVO                               tabla personas
  codigo · nombre · sueldo                codigo (PK) · nombre · sueldo
        ▲                                       ▲
        | extends                               | FOREIGN KEY (codigo)
        |                                       | ON DELETE CASCADE
ProfesorVO                              tabla profesores
  especialidad · horasSemanales           codigo (PK y FK) · especialidad · horas_semanales
```

01

CONSOLIDACIÓN · TEMA 4 — HERENCIA

Listar los profesores con INNER JOIN

NIVEL BÁSICO

Crear la tabla **profesores** en MySQL, la clase **ProfesorVO** que hereda de **PersonaVO** y un listado de profesores que une las dos tablas con `INNER JOIN`. En esta actividad solo se lista: los botones Agregar, Editar y Eliminar del listado de profesores quedan deshabilitados hasta la Actividad 02.

PROYECTO BASE	SQL NUEVO
el de la Práctica 4	FOREIGN KEY · INNER JOIN

// QUÉ CAMBIA EN CADA CAPA

CAPA	ARCHIVO	CAMBIO
Base de datos	`tabla profesores`	Tabla nueva con `codigo` como clave primaria y foránea; datos de prueba
Modelo · VO	`ProfesorVO.java`	**Crear**: `extends PersonaVO`, 2 atributos propios, constructor con `super(...)`
Modelo · DAO	`ProfesorDAO.java`	**Crear** con el método `listarProfesores()`
Controlador	`ProfesorControlador-Servlet.java`	**Crear** en la ruta `/profesor`; por ahora solo lista
Vista	`profesor-lista.jsp` `index.jsp`	**Crear** el listado; agregar en el inicio el enlace Listar Profesores

// TAREAS

1. En **MySQL Workbench**, crear la tabla y cargar los datos. Cada profesor se inserta **primero en personas** (tabla padre) y después en **profesores** (tabla hija): al revés, la clave foránea lo impide.

MySQL Workbench — tabla profesores y datos

```
USE institutoWeb;

-- Tabla hija: codigo es clave primaria y también clave foránea
CREATE TABLE profesores (
    codigo          INT PRIMARY KEY,
    especialidad    VARCHAR(30) NOT NULL,
    horas_semanales INT NOT NULL,
    FOREIGN KEY (codigo) REFERENCES personas (codigo) ON DELETE CASCADE
);

-- 1. Primero la persona (tabla padre)...
INSERT INTO personas (codigo, nombre, sueldo)
VALUES (10, 'Laura', 52000.00), (11, 'Martín', 48500.00), (12, 'Sofía', 61000.00);

-- 2. ...después sus datos de profesor (tabla hija)
INSERT INTO profesores (codigo, especialidad, horas_semanales)
VALUES (10, 'Matemática', 20), (11, 'Programación', 16), (12, 'Física', 24);

-- 3. Comprobar: la consulta que va a usar el DAO
SELECT p.codigo, p.nombre, p.sueldo, pr.especialidad, pr.horas_semanales
FROM personas p
INNER JOIN profesores pr ON p.codigo = pr.codigo;
```

2. Crear **ProfesorVO.java** en el paquete `modelo.vo`: `public class ProfesorVO extends PersonaVO`, con los atributos `especialidad` (`String`) y `horasSemanales` (`int`), un constructor vacío, un constructor de 5 parámetros que empiece con `super(codigo, nombre, sueldo)`, y los getters y setters de los dos atributos propios.

3. Crear **ProfesorDAO.java** en `modelo.dao` con el método `listarProfesores()`, que devuelve `List<ProfesorVO>`. Aplicar las 5 fases tomando como modelo `getLista()` de `PersonaDAO`, con la consulta del `INNER JOIN` del paso 1.

4 Crear **ProfesorControladorServlet.java** en `controlador` con `@WebServlet("/profesor")`. Por ahora el `doGet` siempre lista: guarda `dao.listarProfesores()` con `request.setAttribute("profesores", ...)` y hace `forward` a `/WEB-INF/vista/profesor-lista.jsp`.

5 Crear **profesor-lista.jsp** a partir de una copia de `persona-lista.jsp`: título "Listado de Profesores", columnas Código, Nombre, Sueldo, Especialidad, Horas semanales y Acciones, `<c:forEach var="pr" items="${requestScope.profesores}">`, `colspan` 6 y los botones deshabilitados como en la Práctica 3. Agregar debajo un enlace "Ver personas" a `/persona`.

6 En **index.jsp**: agregar el enlace **Listar Profesores** con `href="profesor"`.

// REFERENCIAS DE CÓDIGO

ProfesorVO.java — herencia y constructor

```java
package modelo.vo;

public class ProfesorVO extends PersonaVO {
    private String especialidad;
    private int horasSemanales;

    public ProfesorVO() {
        super();
    }

    public ProfesorVO(int codigo, String nombre, double sueldo,
                      String especialidad, int horasSemanales) {
        super(codigo, nombre, sueldo);        // los datos de persona los guarda PersonaVO
        this.especialidad = especialidad;
        this.horasSemanales = horasSemanales;
    }

    // getters y setters de especialidad y horasSemanales
}
```

ProfesorDAO.java — SQL y fase 5 de listarProfesores()

```java
// FASE 1: Definir la sentencia SQL (INNER JOIN une las tablas por el código)
String sql = "SELECT p.codigo, p.nombre, p.sueldo, pr.especialidad, pr.horas_semanales "
           + "FROM personas p "
           + "INNER JOIN profesores pr ON p.codigo = pr.codigo";

// FASE 5: Leer el ResultSet
while (rs.next()) {
    lista.add(new ProfesorVO(
            rs.getInt("codigo"),
            rs.getString("nombre"),
            rs.getDouble("sueldo"),
            rs.getString("especialidad"),
            rs.getInt("horas_semanales")));
}
```

profesor-lista.jsp — celdas de cada fila

```
<c:forEach var="pr" items="${requestScope.profesores}">
<tr>
  <td>${pr.codigo}</td>
  <td>${pr.nombre}</td>
  <td>${pr.sueldo}</td>
  <td>${pr.especialidad}</td>
  <td>${pr.horasSemanales}</td>
  ...
```

UN PROFESOR TAMBIÉN ES UNA PERSONA

Laura, Martín y Sofía aparecen también en el listado de `/persona`, porque `getLista()` lee la tabla `personas`, donde están todos. Al revés no pasa: Juan, María y Carlos son personas pero no profesores, así que el `INNER JOIN` no los trae. En la vista, `${pr.nombre}` funciona aunque `ProfesorVO` no declare `getNombre()`: lo hereda de `PersonaVO`.

// CRITERIOS DE VERIFICACIÓN

- [ ] En Workbench, el `SELECT` con `INNER JOIN` devuelve los 3 profesores con sus 5 columnas.
- [ ] En `/profesor`, la aplicación muestra los mismos 3 profesores, con especialidad y horas semanales.
- [ ] Los profesores aparecen también en el listado de personas; las personas que no son profesores no aparecen en el de profesores.
- [ ] El listado de personas sigue funcionando como al final de la Práctica 4.

02

APLICACIÓN · TEMA 4 — TRANSACCIONES

Alta, edición y baja de profesores con transacciones

NIVEL INTERMEDIO

Completar el CRUD de profesores. Agregar y actualizar un profesor obliga a escribir en **dos tablas**, así que esas operaciones se hacen dentro de una **transacción**: se guardan las dos o no se guarda ninguna. Para eliminar alcanza con borrar la persona: `ON DELETE CASCADE` borra la fila de `profesores`.

PROYECTO BASE	SQL NUEVO
el de la Actividad 01	2 × INSERT / UPDATE · CASCADE

// CONCEPTO: TRANSACCIÓN EN JDBC

INSTRUCCIÓN	QUÉ HACE
`con.setAutoCommit(false)`	Inicia la transacción: los cambios quedan pendientes en lugar de guardarse uno por uno
`con.commit()`	Confirma y guarda todos los cambios pendientes, cuando las dos sentencias funcionaron
`con.rollback()`	Deshace todos los cambios pendientes, si alguna sentencia falló

Sin transacción, si el `INSERT` en `personas` funciona y el de `profesores` falla, queda guardada una persona sin sus datos de profesor.

// TAREAS

1. En **ProfesorDAO.java**: agregar `obtenerProfesorPorCodigo(int codigo)`, con la consulta del `INNER JOIN` más `WHERE p.codigo = ?`. Igual que `obtenerXCodigo()` de personas: `ResultSet` en try anidado e `if (rs.next())`.

2. En **ProfesorDAO.java**: agregar `agregarProfesor(ProfesorVO profesor)`, que devuelve `boolean`. Dentro de una transacción, ejecutar primero el `INSERT` en `personas` (código, nombre, sueldo) y después el `INSERT` en `profesores` (código, especialidad, horas). `commit()` si los dos funcionan; `rollback()` en el `catch`.

3. En **ProfesorDAO.java**: agregar `actualizarProfesor(ProfesorVO profesor)` con el mismo esquema y dos `UPDATE`: `nombre` y `sueldo` en `personas`; `especialidad` y `horas_semanales` en `profesores`. Confirmar solo si los dos `executeUpdate()` devuelven más de 0.

4. En **ProfesorDAO.java**: agregar `eliminarProfesor(int codigo)` con un solo `DELETE FROM personas WHERE codigo = ?`.

5. En **ProfesorControladorServlet.java**, con la misma estructura que `PersonaControladorServlet`: en el `doGet`, los `case` `"agregar"` (muestra el formulario) y `"cargarEditar"`; en el `doPost`, `"agregar"`, `"editar"` y `"eliminar"`, todos terminando con redirección (PRG). Antes de agregar, comprobar el código con `PersonaDAO.obtenerXCodigo()` (ver el recuadro).

6 Crear **profesor-form.jsp** y **profesor-editar.jsp** a partir de los de persona, con los 5 campos: `codigo`, `nombre`, `sueldo`, `especialidad` y `horasSemanales`. En **profesor-lista.jsp**, habilitar los botones con `action` hacia `/profesor`.

// REFERENCIAS DE CÓDIGO

ProfesorDAO.java — agregarProfesor() con transacción

```java
public boolean agregarProfesor(ProfesorVO profesor) {
    boolean exito = false;
    // FASE 1: Definir las sentencias SQL (primero la tabla padre)
    String sql1 = "INSERT INTO personas(codigo, nombre, sueldo) VALUES (?, ?, ?)";
    String sql2 = "INSERT INTO profesores(codigo, especialidad, horas_semanales) VALUES (?, ?, ?)";

    // FASES 2 y 3: conexión y los dos PreparedStatement
    try (Connection con = Conexion.obtener();
         PreparedStatement ps1 = con.prepareStatement(sql1);
         PreparedStatement ps2 = con.prepareStatement(sql2)) {

        con.setAutoCommit(false);               // INICIO DE LA TRANSACCIÓN
        try {
            // FASE 3: Asignar parámetros  |  FASE 4: Ejecutar
            ps1.setInt(1, profesor.getCodigo());
            ps1.setString(2, profesor.getNombre());
            ps1.setDouble(3, profesor.getSueldo());
            ps1.executeUpdate();                // INSERT en personas

            ps2.setInt(1, profesor.getCodigo());
            ps2.setString(2, profesor.getEspecialidad());
            ps2.setInt(3, profesor.getHorasSemanales());
            ps2.executeUpdate();                // INSERT en profesores

            con.commit();                       // los dos funcionaron → guardar
            exito = true;
        } catch (SQLException e) {
            con.rollback();                     // alguno falló → no queda ninguno
            throw e;                            // lo informa el catch de afuera
        }
    } catch (SQLException e) {
        System.err.println("Error al agregar profesor: " + e.getMessage());
    }
    return exito;
}
```

ProfesorControladorServlet.java — comprobación del código al agregar

```java
private final ProfesorDAO dao = new ProfesorDAO();
private final PersonaDAO personaDAO = new PersonaDAO();   // busca en TODAS las personas

// dentro de agregar(), después de leer los 5 parámetros del formulario:
if (personaDAO.obtenerXCodigo(profesor.getCodigo()) != null) {
    request.setAttribute("error",
            "Error: El código " + profesor.getCodigo() + " ya está registrado. Ingrese uno diferente.");
    request.setAttribute("profesor", profesor);           // mantener los datos
    request.getRequestDispatcher("/WEB-INF/vista/profesor-form.jsp").forward(request, response);
    return;
}
dao.agregarProfesor(profesor);
response.sendRedirect(request.getContextPath() + "/profesor");   // PRG
```

> **¿POR QUÉ COMPROBAR EL CÓDIGO CON PERSONADAO?**
>
> Si se intenta agregar un profesor con el código 1 (Juan, que es persona pero no profesor), `obtenerProfesorPorCodigo(1)` devuelve `null`, porque Juan no está en `profesores`. La comprobación pasaría, el `INSERT` en `personas` fallaría por clave repetida y la transacción se revertiría sin ningún mensaje. `obtenerXCodigo()` de `PersonaDAO` busca en `personas`, donde están todos los códigos usados.

// PRUEBA DE LA TRANSACCIÓN

Para ver el `rollback()` en acción: cambiar a propósito el nombre de la tabla en `sql2` (por ejemplo, `profesoress`), compilar e intentar agregar un profesor. El segundo `INSERT` falla y, al revisar en Workbench, la persona **tampoco** quedó en `personas`. Después, volver a dejar el nombre correcto.

// CRITERIOS DE VERIFICACIÓN

- [ ] Al agregar un profesor, aparece en `/profesor` y en `/persona`, y en Workbench tiene su fila en las dos tablas.
- [ ] Al usar el código de una persona existente (por ejemplo, 1), se muestra el mensaje de error y no se inserta nada en ninguna tabla.
- [ ] Al editar sueldo y especialidad, el cambio se ve en el listado y en las dos tablas de MySQL.
- [ ] Al eliminar un profesor, desaparece de las dos tablas y también del listado de personas.
- [ ] Con el nombre de tabla mal escrito en `sql2`, no queda guardado nada en `personas`.

03

DESAFÍO · TEMA 4 — HERENCIA PROPIA

Una `subclase` de la entidad propia

NIVEL AVANZADO

Retomar el proyecto de la **Actividad 03 de la Práctica 4** (el CRUD de la entidad propia en MySQL) y agregarle una **subclase** que tenga sentido, con su propia tabla en MySQL unida por el código. Implementar su CRUD completo: listado con `INNER JOIN`, alta y edición con transacción, y baja con `ON DELETE CASCADE`. Los ejemplos usan **ProductoImportado**, subclase de **Producto**: reemplazar por la subclase elegida.

PROYECTO BASE	REFERENCIA
Act. 03 · Práctica 4	**ProfesorDAO de las Act. 01 y 02**

// IDEAS DE SUBCLASE

ENTIDAD	SUBCLASE	ATRIBUTOS PROPIOS (EJEMPLO)
Producto	`ProductoImportado`	`paisOrigen` (String), `costoEnvio` (double)
Libro	`LibroDigital`	`formato` (String), `tamanioMB` (double)
Película	`PeliculaEstreno`	`sala` (int), `precioEntrada` (double)
Videojuego	`VideojuegoOnline`	`servidor` (String), `maxJugadores` (int)
Estudiante	`EstudianteBecado`	`tipoBeca` (String), `porcentaje` (int)

Cualquier otra subclase sirve si cumple la regla de la herencia: se puede decir "un *ProductoImportado* **es un** *Producto*". Acordarla con la docente.

// TAREAS

1 En **MySQL Workbench**: crear la tabla hija con `codigo` como clave primaria y foránea, con `ON DELETE CASCADE`, y cargar al menos 3 registros (primero en la tabla padre). Guardar el script como `subclase.sql`.

MySQL Workbench — tabla hija (ejemplo con ProductoImportado)

```
USE institutoWeb;

CREATE TABLE productos_importados (
    codigo      INT PRIMARY KEY,
    pais_origen VARCHAR(30) NOT NULL,
    costo_envio DECIMAL(8,2) NOT NULL,
    FOREIGN KEY (codigo) REFERENCES productos (codigo) ON DELETE CASCADE
);

INSERT INTO productos (codigo, nombre, precio, descripcion, stock)
VALUES (20, 'Notebook', 32000.00, 'Pantalla de 14 pulgadas', 5);

INSERT INTO productos_importados (codigo, pais_origen, costo_envio)
VALUES (20, 'China', 1800.00);
```

2. Crear el **VO de la subclase** con `extends`, sus atributos propios y un constructor que llame a `super(...)` con los atributos de la clase padre.

3. Crear el **DAO de la subclase** con los 5 métodos del CRUD, tomando como modelo `ProfesorDAO`: listar y obtener por código con `INNER JOIN`, agregar y actualizar con transacción, eliminar con un solo `DELETE` en la tabla padre.

4. Crear el **Servlet** de la subclase con su propia ruta (por ejemplo, `/importado`). Al agregar, comprobar el código con el `obtenerXCodigo()` del DAO de la **clase padre**.

5. Crear las **tres vistas** (listado, alta y edición) y agregar en `index.jsp` el enlace al listado de la subclase.

// ARCHIVOS A CREAR Y MODIFICAR

Estructura esperada (ejemplo con ProductoImportado)

```
Base de datos:
  tabla productos_importados   ← CREATE TABLE + INSERT (en subclase.sql)

Capa Modelo:
  src/main/java/modelo/vo/ProductoImportadoVO.java    ← crear (extends ProductoVO)
  src/main/java/modelo/dao/ProductoImportadoDAO.java  ← crear (JOIN + transacciones)

Capa Controlador:
  src/main/java/controlador/ProductoImportadoControladorServlet.java  ← crear

Capa Vista:
  src/main/webapp/WEB-INF/vista/importado-lista.jsp    ← crear
  src/main/webapp/WEB-INF/vista/importado-form.jsp     ← crear
  src/main/webapp/WEB-INF/vista/importado-editar.jsp   ← crear
  src/main/webapp/index.jsp                            ← enlace nuevo
```

> **QUÉ CAMBIA Y QUÉ NO**
>
> La clase padre, su DAO y su Servlet no se tocan: la herencia permite agregar un tipo nuevo sin modificar lo que ya funciona. Lo único que la subclase toma de la clase padre es lo que hereda en Java (`super(...)` y los getters) y la fila de la tabla padre en MySQL.

// CRITERIOS DE VERIFICACIÓN

- [ ] En Workbench, el `INNER JOIN` entre las dos tablas devuelve al menos 3 registros de la subclase.
- [ ] El listado de la subclase muestra los atributos heredados y los propios.
- [ ] Agregar y editar escriben en las dos tablas; si una sentencia falla, no queda guardado nada (rollback).
- [ ] Al eliminar, el registro desaparece de las dos tablas gracias a `ON DELETE CASCADE`.
- [ ] La entrega incluye `subclase.sql` con el `CREATE TABLE` y los `INSERT`.